In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Ustawienia dla polskich znaków
plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
```

Wstęp do pandas

`Pandas` to biblioteka w Pythonie, która umożliwia łatwe i efektywne manipulowanie oraz analizowanie danych.

Jest szczególnie przydatna do pracy z danymi tabelarycznymi, podobnymi do tych w arkuszach kalkulacyjnych.

CZĘŚĆ 1: Tworzenie DataFrame z polskimi danymi

In [ ]:
```
# Przykład 1: Dane o polskich miastach
data_miasta = {
    'Miasto': ['Warszawa', 'Kraków', 'Łódź', 'Wrocław', 'Poznań', 'Gdańsk', 'Szc
    'Województwo': ['mazowieckie', 'małopolskie', 'łódzkie', 'dolnośląskie',
                    'wielkopolskie', 'pomorskie', 'zachodniopomorskie'],
    'Populacja': [1863056, 779966, 672185, 643782, 532048, 470907, 395513],
    'Powierzchnia_km2': [517, 327, 293, 293, 262, 262, 301],
    'Rok_nadania_praw': [1413, 1257, 1423, 1242, 1253, 1263, 1243]
}

miasta = pd.DataFrame(data_miasta)
print(miasta)
```

```
     Miasto         Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa         mazowieckie    1863056               517              1413
1    Kraków         małopolskie     779966               327              1257
2      Łódź             łódzkie     672185               293              1423
3   Wrocław        dolnośląskie     643782               293              1242
4    Poznań       wielkopolskie     532048               262              1253
5    Gdańsk           pomorskie     470907               262              1263
6  Szczecin  zachodniopomorskie     395513               301              1243
```

CZĘŚĆ 2: PODSTAWOWE METODY INSPEKCJI DANYCH

In [ ]:
```
print(">>> .head() - Pierwsze 5 wierszy (lub inna liczba)")
print(miasta.head(3))
```

```
>>> .head() - Pierwsze 5 wierszy (lub inna liczba)
     Miasto  Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa  mazowieckie    1863056               517              1413
1    Kraków  małopolskie     779966               327              1257
2      Łódź      łódzkie     672185               293              1423
```

In [ ]:
```
print(">>> .info() - Informacje o strukturze danych")
miasta.info()
```

```
>>> .info() - Informacje o strukturze danych
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Miasto            7 non-null      object
 1   Województwo       7 non-null      object
 2   Populacja         7 non-null      int64
 3   Powierzchnia_km2  7 non-null      int64
 4   Rok_nadania_praw  7 non-null      int64
dtypes: int64(3), object(2)
memory usage: 412.0+ bytes
```

In [ ]:
```
print(">>> .shape - Wymiary DataFrame (wiersze, kolumny)")
print(f"Nasz DataFrame ma: {miasta.shape[0]} wierszy i {miasta.shape[1]} kolumn"
```

```
>>> .shape - Wymiary DataFrame (wiersze, kolumny)
Nasz DataFrame ma: 7 wierszy i 5 kolumn
```

In [ ]:
```
print(">>> .describe() - Statystyki opisowe dla kolumn numerycznych")
print(miasta.describe())
```

```
>>> .describe() - Statystyki opisowe dla kolumn numerycznych
          Populacja  Powierzchnia_km2  Rok_nadania_praw
count  7.000000e+00          7.000000          7.000000
mean   7.653510e+05        322.142857       1299.142857
std    5.010882e+05         88.856492         81.581102
min    3.955130e+05        262.000000       1242.000000
25%    5.014775e+05        277.500000       1248.000000
50%    6.437820e+05        293.000000       1257.000000
75%    7.260755e+05        314.000000       1338.000000
max    1.863056e+06        517.000000       1423.000000
```

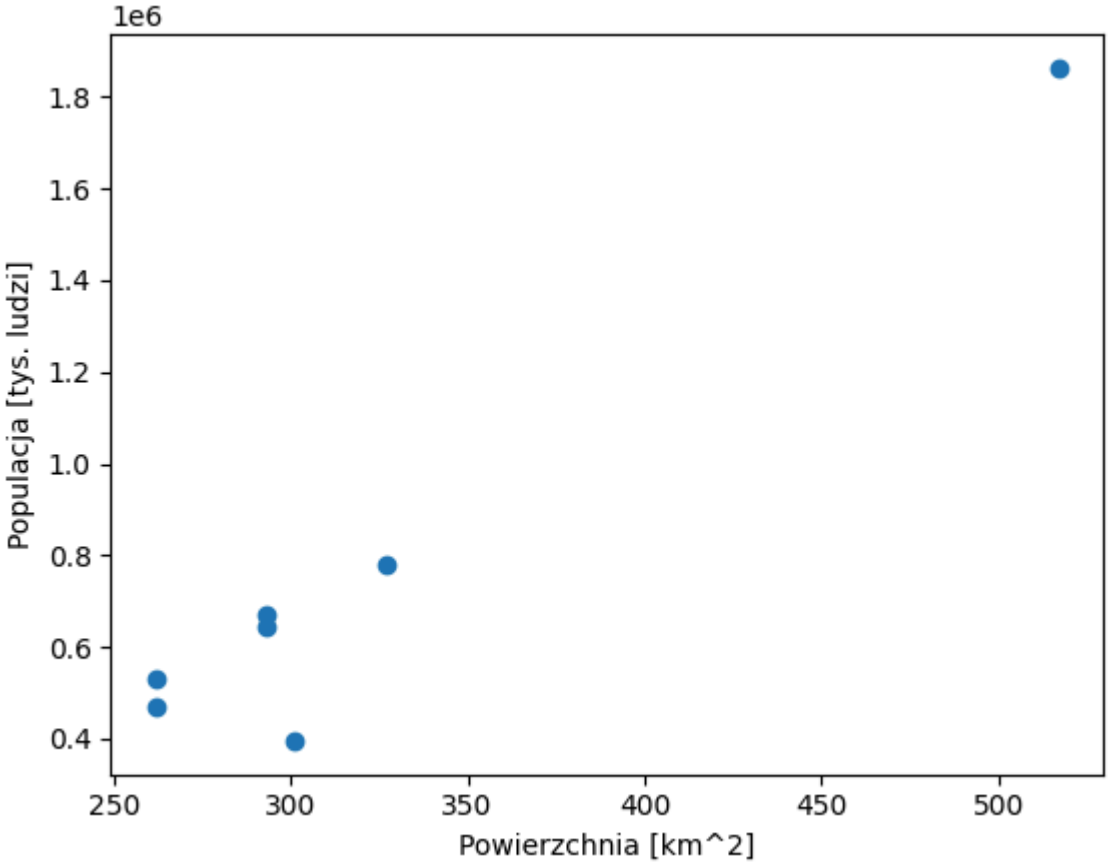
In [ ]:
```
print(">>> .columns - Nazwy kolumn")
print(miasta.columns.tolist())
```

```
>>> .columns - Nazwy kolumn
['Miasto', 'Województwo', 'Populacja', 'Powierzchnia_km2', 'Rok_nadania_praw']
```

In [ ]:
```
print(">>> .index - Indeks wierszy")
print(miasta.index)
```

```
>>> .index - Indeks wierszy
RangeIndex(start=0, stop=7, step=1)
```

In [ ]:
```
plt.scatter(miasta["Powierzchnia_km2"], miasta["Populacja"])
plt.xlabel("Powierzchnia [km^2]")
plt.ylabel("Populacja [tys. ludzi]")
plt.show()
```



CZĘŚĆ 3: SORTOWANIE DANYCH

In [ ]:
```
print("1. Sortowanie rosnąco po populacji:")
print(miasta.sort_values("Populacja").head())
```

```
1. Sortowanie rosnąco po populacji:
     Miasto         Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
6  Szczecin  zachodniopomorskie     395513               301              1243
5    Gdańsk           pomorskie     470907               262              1263
4    Poznań       wielkopolskie     532048               262              1253
3   Wrocław        dolnośląskie     643782               293              1242
2      Łódź             łódzkie     672185               293              1423
```

In [ ]:
```
print("2. Sortowanie malejąco po populacji:")
print(miasta.sort_values("Populacja", ascending=False).head())
```

```
2. Sortowanie malejąco po populacji:
     Miasto    Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa    mazowieckie    1863056               517              1413
1    Kraków    małopolskie     779966               327              1257
2      Łódź        łódzkie     672185               293              1423
3   Wrocław   dolnośląskie     643782               293              1242
4    Poznań  wielkopolskie     532048               262              1253
```

In [ ]:
```
print("3. Sortowanie według wielu kolumn:")
print("   (Województwo rosnąco, potem Populacja malejąco)")
print(miasta.sort_values(["Województwo", "Populacja"],
                         ascending=[True, False]))
```

```
3. Sortowanie według wielu kolumn:
   (Województwo rosnąco, potem Populacja malejąco)
     Miasto         Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
3   Wrocław        dolnośląskie     643782               293              1242
0  Warszawa         mazowieckie    1863056               517              1413
1    Kraków         małopolskie     779966               327              1257
5    Gdańsk           pomorskie     470907               262              1263
4    Poznań       wielkopolskie     532048               262              1253
6  Szczecin  zachodniopomorskie     395513               301              1243
2      Łódź             łódzkie     672185               293              1423
```

CZĘŚĆ 4: WYBIERANIE KOLUMN

In [ ]:
```python
print("1. Jedna kolumna (zwraca Series):")
print(miasta["Miasto"])
print(f"\nTyp: {type(miasta['Miasto'])}")
```

```
1. Jedna kolumna (zwraca Series):
0    Warszawa
1      Kraków
2        Łódź
3     Wrocław
4      Poznań
5      Gdańsk
6    Szczecin
Name: Miasto, dtype: object

Typ: <class 'pandas.core.series.Series'>
```

In [ ]:
```python
print("2. Wiele kolumn (zwraca DataFrame):")
print(miasta[["Miasto", "Populacja"]])
```

```
2. Wiele kolumn (zwraca DataFrame):
     Miasto  Populacja
0  Warszawa    1863056
1    Kraków     779966
2      Łódź     672185
3   Wrocław     643782
4    Poznań     532048
5    Gdańsk     470907
6  Szczecin     395513
```

In [ ]:
```python
print("3. Używanie zmiennej z listą kolumn:")
kolumny_do_wyswietlenia = ["Miasto",
                           "Województwo",
                           "Populacja"]
print(miasta[kolumny_do_wyswietlenia])
```

```
3. Używanie zmiennej z listą kolumn:
     Miasto         Województwo  Populacja
0  Warszawa         mazowieckie    1863056
1    Kraków         małopolskie     779966
2      Łódź             łódzkie     672185
3   Wrocław        dolnośląskie     643782
4    Poznań       wielkopolskie     532048
5    Gdańsk           pomorskie     470907
6  Szczecin  zachodniopomorskie     395513
```

CZĘŚĆ 5: FILTROWANIE WIERSZY

In [ ]:
```
print("1. Miasta z populacją powyżej 600 000:")
print(miasta[miasta["Populacja"] > 600000])
```

```
1. Miasta z populacją powyżej 600 000:
     Miasto   Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa   mazowieckie    1863056               517              1413
1    Kraków   małopolskie     779966               327              1257
2      Łódź       łódzkie     672185               293              1423
3   Wrocław  dolnośląskie     643782               293              1242
```

In [ ]:
```
print("2. Miasta z województwa mazowieckiego:")
print(miasta[miasta["Województwo"] == "mazowieckie"])
```

```
2. Miasta z województwa mazowieckiego:
     Miasto  Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa  mazowieckie    1863056               517              1413
```

In [ ]:
```
print("3. Miasta z prawami miejskimi przed 1300 rokiem:")
print(miasta[miasta["Rok_nadania_praw"] < 1300])
```

```
3. Miasta z prawami miejskimi przed 1300 rokiem:
     Miasto         Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
1    Kraków         małopolskie     779966               327              1257
3   Wrocław        dolnośląskie     643782               293              1242
4    Poznań       wielkopolskie     532048               262              1253
5    Gdańsk           pomorskie     470907               262              1263
6  Szczecin  zachodniopomorskie     395513               301              1243
```

In [ ]:
```
print("4. Złożone warunki z operatorem AND (&):")
print("   Miasta powyżej 500k ludności I powierzchnia powyżej 280 km²")
duze_miasta = miasta[(miasta["Populacja"] > 500000) &
                     (miasta["Powierzchnia_km2"] > 280)]
print(duze_miasta)
```

```
4. Złożone warunki z operatorem AND (&):
   Miasta powyżej 500k ludności I powierzchnia powyżej 280 km²
     Miasto   Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa   mazowieckie    1863056               517              1413
1    Kraków   małopolskie     779966               327              1257
2      Łódź       łódzkie     672185               293              1423
3   Wrocław  dolnośląskie     643782               293              1242
```

In [ ]:
```
print("5. Złożone warunki z operatorem OR (|):")
print("   Miasta z Warszawy LUB Krakowa")
stolice = miasta[(miasta["Miasto"] == "Warszawa") |
                 (miasta["Miasto"] == "Kraków")]
print(stolice)
```

```
5. Złożone warunki z operatorem OR (|):
   Miasta z Warszawy LUB Krakowa
     Miasto  Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa  mazowieckie    1863056               517              1413
1    Kraków  małopolskie     779966               327              1257
```

In [ ]:
```
print("6. Używanie .isin() dla wielu wartości:")
wybrane_miasta = ["Warszawa", "Kraków", "Wrocław"]
print(miasta[miasta["Miasto"].isin(wybrane_miasta)])
```

```
6. Używanie .isin() dla wielu wartości:
     Miasto   Województwo  Populacja  Powierzchnia_km2  Rok_nadania_praw
0  Warszawa   mazowieckie    1863056               517              1413
1    Kraków   małopolskie     779966               327              1257
3   Wrocław  dolnośląskie     643782               293              1242
```

CZĘŚĆ 6: DODAWANIE NOWYCH KOLUMN

In [ ]:
```python
print("1. Obliczanie gęstości zaludnienia:")
miasta["Gęstość_na_km2"] = miasta["Populacja"] / miasta["Powierzchnia_km2"]
print(miasta[["Miasto", "Populacja", "Powierzchnia_km2", "Gęstość_na_km2"]])
```

```
1. Obliczanie gęstości zaludnienia:
     Miasto  Populacja  Powierzchnia_km2  Gęstość_na_km2
0  Warszawa    1863056               517     3603.589942
1    Kraków     779966               327     2385.217125
2      Łódź     672185               293     2294.146758
3   Wrocław     643782               293     2197.208191
4    Poznań     532048               262     2030.717557
5    Gdańsk     470907               262     1797.354962
6  Szczecin     395513               301     1313.996678
```

In [ ]:
```python
print("2. Kategoryzacja miast według wielkości:")
def kategoryzuj_miasto(populacja):
    if populacja > 1000000:
        return "Bardzo duże"
    elif populacja > 500000:
        return "Duże"
    else:
        return "Średnie"

miasta["Kategoria"] = miasta["Populacja"].apply(kategoryzuj_miasto)
print(miasta[["Miasto", "Populacja", "Kategoria"]])
```

```
2. Kategoryzacja miast według wielkości:
     Miasto  Populacja    Kategoria
0  Warszawa    1863056  Bardzo duże
1    Kraków     779966         Duże
2      Łódź     672185         Duże
3   Wrocław     643782         Duże
4    Poznań     532048         Duże
5    Gdańsk     470907      Średnie
6  Szczecin     395513      Średnie
```

Zadania

In [ ]:
```python
# Dataset do zadań
temperatura = pd.DataFrame({
    'Miasto': ['Warszawa', 'Kraków', 'Gdańsk', 'Wrocław', 'Poznań', 'Łódź'],
    'Styczeń': [-2, -1, 0, -1, -2, -3],
    'Kwiecień': [9, 10, 8, 11, 10, 9],
    'Lipiec': [19, 20, 17, 21, 19, 20],
    'Październik': [9, 9, 10, 10, 9, 8]
})

print("Dataset 'temperatura' - średnie temperatury w polskich miastach:")
print(temperatura)
```

```
Dataset 'temperatura' - średnie temperatury w polskich miastach:
     Miasto  Styczeń  Kwiecień  Lipiec  Październik
0  Warszawa       -2         9      19            9
1    Kraków       -1        10      20            9
2    Gdańsk        0         8      17           10
3   Wrocław       -1        11      21           10
4    Poznań       -2        10      19            9
5      Łódź       -3         9      20            8
```

Zadanie 1: podstawowa inspekcja

Używając DataFrame `temperatura`, wykonaj następujące operacje:

a) Wyświetl pierwsze 3 wiersze

b) Wyświetl informacje o typach danych (.info())

c) Wyświetl kształt DataFrame (.shape)

d) Wyświetl tylko nazwy kolumn (.columns)

In [ ]:
```
# (1a)
print(temperatura.head(3))
```

```
     Miasto  Styczeń  Kwiecień  Lipiec  Październik
0  Warszawa       -2         9      19            9
1    Kraków       -1        10      20            9
2    Gdańsk        0         8      17           10
```

In [ ]:
```
# (1b)
print(temperatura.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Miasto       6 non-null      object
 1   Styczeń      6 non-null      int64
 2   Kwiecień     6 non-null      int64
 3   Lipiec       6 non-null      int64
 4   Październik  6 non-null      int64
dtypes: int64(4), object(1)
memory usage: 372.0+ bytes
None
```

In [ ]:
```
# (1c)
print(temperatura.shape)
```

```
(6, 5)
```

In [ ]:
```
# (1d)
print(temperatura.columns)
```

```
Index(['Miasto', 'Styczeń', 'Kwiecień', 'Lipiec', 'Październik'], dtype='object')
```

Zadanie 2: sortowanie

a) Posortuj miasta według temperatury w lipcu (rosnąco)

b) Posortuj miasta według temperatury w styczniu (malejąco)

c) Posortuj miasta według temperatury w lipcu (malejąco), a następnie według kwietnia (rosnąco)

In [ ]:
```
# (2a)
print(temperatura.sort_values("Lipiec", ascending=True).head())
```

```
    Miasto  Styczeń  Kwiecień  Lipiec  Październik
2   Gdańsk        0         8      17           10
0 Warszawa       -2         9      19            9
4   Poznań       -2        10      19            9
1   Kraków       -1        10      20            9
5     Łódź       -3         9      20            8
```

In [ ]:
```
# (2b)
print(temperatura.sort_values("Styczeń", ascending=False).head())
```

```
    Miasto  Styczeń  Kwiecień  Lipiec  Październik
2   Gdańsk        0         8      17           10
1   Kraków       -1        10      20            9
3  Wrocław       -1        11      21           10
0 Warszawa       -2         9      19            9
4   Poznań       -2        10      19            9
```

In [ ]:
```
# (2c)
print(temperatura.sort_values(["Lipiec", "Kwiecień"],
                              ascending=[False, True]))
```

```
    Miasto  Styczeń  Kwiecień  Lipiec  Październik
3  Wrocław       -1        11      21           10
5     Łódź       -3         9      20            8
1   Kraków       -1        10      20            9
0 Warszawa       -2         9      19            9
4   Poznań       -2        10      19            9
2   Gdańsk        0         8      17           10
```

Zadanie 3: Wybieranie kolumn

a) Wybierz tylko kolumnę 'Miasto' (wynik: Series)

b) Wybierz kolumny 'Miasto' i 'Lipiec' (wynik: DataFrame)

c) Wybierz kolumny 'Miasto', 'Styczeń' i 'Lipiec', używając zmiennej z listą nazw kolumn

In [ ]:
```
# (3a)
print(temperatura["Miasto"])
```

```
0    Warszawa
1      Kraków
2      Gdańsk
3     Wrocław
4      Poznań
5        Łódź
Name: Miasto, dtype: object
```

In [ ]:
```
# (3b)
print(temperatura[["Miasto","Lipiec"]])
```

```
    Miasto  Lipiec
0  Warszawa      19
1    Kraków      20
2    Gdańsk      17
3   Wrocław      21
4    Poznań      19
5      Łódź      20
```

In [ ]:
```
# (3c)
kolumny_do_wyswietlenia = ["Miasto","Styczeń","Lipiec"]
print(temperatura[kolumny_do_wyswietlenia])
```

```
    Miasto  Styczeń  Lipiec
0  Warszawa       -2      19
1    Kraków       -1      20
2    Gdańsk        0      17
3   Wrocław       -1      21
4    Poznań       -2      19
5      Łódź       -3      20
```

Zadanie 4: Filtrowanie

a) Znajdź miasta, gdzie temperatura w lipcu jest większa niż 19 stopni

b) Znajdź miasta, gdzie temperatura w styczniu jest większa lub równa -1

c) Znajdź miasta, gdzie temperatura w kwietniu wynosi dokładnie 10 stopni

d) Znajdź miasta, gdzie temperatura w lipcu jest 20 LUB temperatura w styczniu jest -1

In [ ]:
```
# (4a)
print(temperatura[temperatura["Lipiec"] > 19])
```

```
   Miasto  Styczeń  Kwiecień  Lipiec  Październik
1  Kraków       -1        10      20            9
3 Wrocław       -1        11      21           10
5    Łódź       -3         9      20            8
```

In [ ]:
```
# (4b)
print(temperatura[temperatura["Styczeń"] >= -1])
```

```
   Miasto  Styczeń  Kwiecień  Lipiec  Październik
1  Kraków       -1        10      20            9
2  Gdańsk        0         8      17           10
3 Wrocław       -1        11      21           10
```

In [ ]:
```
# (4c)
print(temperatura[temperatura["Kwiecień"] == 10])
```

```
  Miasto  Styczeń  Kwiecień  Lipiec  Październik
1 Kraków       -1        10      20            9
4 Poznań       -2        10      19            9
```

In [ ]:
```
# (4d)
print(temperatura[(temperatura["Lipiec"] == 20) | (temperatura["Styczeń"] == -1)
```

```
   Miasto  Styczeń  Kwiecień  Lipiec  Październik
1  Kraków       -1        10      20            9
3 Wrocław       -1        11      21           10
5    Łódź       -3         9      20            8
```

Zadanie 5: Złożone filtrowanie

a) Znajdź miasta, gdzie temperatura w lipcu > 18 ORAZ w styczniu >= -2

b) Znajdź miasta: 'Warszawa', 'Kraków', 'Gdańsk'

c) Znajdź miasta, gdzie temperatura w lipcu jest między 18 a 20 stopni (włącznie)

In [ ]:
```python
# (5a)
print(temperatura[(temperatura["Lipiec"] > 18) & (temperatura["Styczeń"] >= -2)]
```

```
   Miasto  Styczeń  Kwiecień  Lipiec  Październik
0 Warszawa       -2         9      19            9
1  Kraków       -1        10      20            9
3 Wrocław       -1        11      21           10
4  Poznań       -2        10      19            9
```

In [ ]:
```python
# (5b)
wybrane_miasta = ["Warszawa", "Kraków", "Gdańsk"]
print(temperatura[temperatura["Miasto"].isin(wybrane_miasta)])
```

```
   Miasto  Styczeń  Kwiecień  Lipiec  Październik
0 Warszawa       -2         9      19            9
1  Kraków       -1        10      20            9
2  Gdańsk        0         8      17           10
```

In [ ]:
```python
# (5c)
print(temperatura[(temperatura["Lipiec"] >= 18) & (temperatura["Lipiec"] <= 20)]
```

```
   Miasto  Styczeń  Kwiecień  Lipiec  Październik
0 Warszawa       -2         9      19            9
1  Kraków       -1        10      20            9
4  Poznań       -2        10      19            9
5    Łódź       -3         9      20            8
```

Zadanie 6: Dodawanie nowych kolumn

a) Dodaj kolumnę 'Średnia_roczna' - średnia z wszystkich miesięcy

b) Dodaj kolumnę 'Amplituda' - różnica między lipcem a styczniem

c) Dodaj kolumnę 'Ciepłe_miasto' (True jeśli średnia roczna > 11, False w przeciwnym razie)

In [ ]:
```python
# (6a)
temperatura["Średnia_roczna"] = (temperatura["Styczeń"] + temperatura["Kwiecień"
print(temperatura[["Miasto","Średnia_roczna"]])
```

```
    Miasto  Średnia_roczna
0  Warszawa            8.75
1    Kraków            9.50
2    Gdańsk            8.75
3   Wrocław           10.25
4    Poznań            9.00
5      Łódź            8.50
```

In [ ]:
```python
# (6b)
temperatura["Aplituda"] = (temperatura["Lipiec"] - temperatura["Styczeń"])
print(temperatura[["Miasto","Aplituda"]])
```

```
    Miasto  Aplituda
0  Warszawa        21
1    Kraków        21
2    Gdańsk        17
3   Wrocław        22
4    Poznań        21
5      Łódź        23
```

In [ ]:
```python
# (6c)
temperatura["Ciepłe_miasto"] = temperatura["Średnia_roczna"] > 11
print(temperatura[["Miasto", "Średnia_roczna", "Ciepłe_miasto"]])
```

```
    Miasto  Średnia_roczna  Ciepłe_miasto
0  Warszawa            8.75          False
1    Kraków            9.50          False
2    Gdańsk            8.75          False
3   Wrocław           10.25          False
4    Poznań            9.00          False
5      Łódź            8.50          False
```

Zadanie: zaawansowane

```
# Nowy dataset dla zaawansowanego zadania
wyniki_egzaminy = pd.DataFrame({
    'Student': ['Anna', 'Bartek', 'Celina', 'Damian', 'Ewa',
'Filip', 'Gabriela', 'Hubert'],
    'Matematyka': [85, 92, 78, 88, 95, 72, 89, 81],
    'Fizyka': [78, 88, 82, 75, 91, 68, 85, 79],
    'Informatyka': [92, 85, 88, 90, 89, 95, 82, 87],
    'Kierunek': ['Informatyka', 'Fizyka', 'Matematyka',
'Informatyka',
                 'Matematyka', 'Informatyka', 'Fizyka',
'Matematyka']
})
```

a) Dodaj kolumnę 'Średnia' z wyników z trzech przedmiotów

b) Dodaj kolumnę 'Status' - 'Zaliczony' jeśli średnia >= 85, 'Niezaliczony' w przeciwnym razie

c) Dla każdego kierunku znajdź studenta z najwyższą średnią:

- Przefiltruj po kierunku
- Posortuj według średniej malejąco
- Weź pierwszy wiersz

- Wykonaj dla każdego z 3 kierunków

d) Znajdź wszystkich studentów informatyki, którzy mają średnią >= 85 i wynik z informatyki >= 90, posortowanych według średniej malejąco

In [ ]:
```
wyniki_egzaminy = pd.DataFrame({
    'Student': ['Anna', 'Bartek', 'Celina', 'Damian', 'Ewa', 'Filip', 'Gabriela'
    'Matematyka': [85, 92, 78, 88, 95, 72, 89, 81],
    'Fizyka': [78, 88, 82, 75, 91, 68, 85, 79],
    'Informatyka': [92, 85, 88, 90, 89, 95, 82, 87],
    'Kierunek': ['Informatyka', 'Fizyka', 'Matematyka', 'Informatyka',
                 'Matematyka', 'Informatyka', 'Fizyka', 'Matematyka']
})
```

In [ ]:
```
student = pd.DataFrame(wyniki_egzaminy)
```

In [ ]:
```
student.head()
```

Out[ ]:

	Student	Matematyka	Fizyka	Informatyka	Kierunek
0	Anna	85	78	92	Informatyka
1	Bartek	92	88	85	Fizyka
2	Celina	78	82	88	Matematyka
3	Damian	88	75	90	Informatyka
4	Ewa	95	91	89	Matematyka

In [ ]:
```
# (zaaw-a)
student["Średnia"] = (student["Informatyka"] + student["Fizyka"] + student["Mate
print(student[["Student","Średnia"]])
```

```
   Student    Średnia
0     Anna  85.000000
1   Bartek  88.333333
2   Celina  82.666667
3   Damian  84.333333
4      Ewa  91.666667
5    Filip  78.333333
6 Gabriela  85.333333
7   Hubert  82.333333
```

In [ ]:
```
# (zaaw-b)
import numpy as np
student["Status"] = np.where(student["Średnia"] >= 85, "Zaliczony", "Niezaliczon
print(student[["Student", "Średnia", "Status"]])
```

```
   Student    Średnia        Status
0     Anna  85.000000     Zaliczony
1   Bartek  88.333333     Zaliczony
2   Celina  82.666667  Niezaliczony
3   Damian  84.333333  Niezaliczony
4      Ewa  91.666667     Zaliczony
5    Filip  78.333333  Niezaliczony
6 Gabriela  85.333333     Zaliczony
7   Hubert  82.333333  Niezaliczony
```

In [ ]:
```
# (zaaw-c)
wyniki_egzaminy["Średnia"] = (wyniki_egzaminy["Informatyka"] + wyniki_egzaminy["
kierunki = ['Informatyka', 'Fizyka', 'Matematyka']
print("Najlepsi studenci na poszczególnych kierunkach:\n")

for k in kierunki:
    filtr = student[student['Kierunek'] == k]
    najlepszy = filtr.sort_values(by='Średnia', ascending=False).head(1)
    print(f"Kierunek: {k}")
    print(najlepszy[['Student', 'Średnia']])
    print("-" * 30)
```

```
Najlepsi studenci na poszczególnych kierunkach:

Kierunek: Informatyka
  Student  Średnia
0    Anna     85.0
------------------------------
Kierunek: Fizyka
  Student    Średnia
1  Bartek  88.333333
------------------------------
Kierunek: Matematyka
  Student    Średnia
4     Ewa  91.666667
------------------------------
```

In [51]:
```
# (zaaw-d)
studenci_informatyki = wyniki_egzaminy[
    (wyniki_egzaminy['Kierunek'] == 'Informatyka') &
    (wyniki_egzaminy['Średnia'] >= 85) &
    (wyniki_egzaminy['Informatyka'] >= 90)
].sort_values(by='Średnia', ascending=False)

print("Studenci informatyki spełniający kryteria (posortowani malejąco po średni
print(studenci_informatyki[['Student', 'Kierunek', 'Informatyka', 'Średnia']])
```

```
Studenci informatyki spełniający kryteria (posortowani malejąco po średniej):
  Student     Kierunek  Informatyka  Średnia
0    Anna  Informatyka           92     85.0
```